

Final Capstone Report

Capstone Project, Postgraduate Diploma in Artificial Intelligence and Machine Learning

September 2026

Final Capstone Report

Help Desk Ticket SLA and Volume Forecasting

Validation status: all eight notebooks completed with the actual project `.venv` kernel after the recursive-calendar correction. The local final suite passed **39/39 tests in a clean LOCAL submission copy** using the existing `.venv`, including FastAPI `TestClient` checks. The copy excluded raw CSV, `mlflow.db`, and `editor/cache/archive` folders. This was not a Git clone, fresh dependency installation, Docker or cloud CI run. Validation selects **XGBoost at 7 days** and **ETS at the primary 30-day horizon**. The earlier SciPy DLL block no longer reproduces; no policy bypass was used. Docker and GitHub Actions were not executed; GitHub publication is not complete.

An actual localhost HTTP demonstration also passed: health returned OK, seven ETS forecast points started on 2023-03-15, and an invalid horizon returned HTTP 422. The temporary server was stopped afterward. The corrected technical/business decks contain 12/10 slides, with no off-slide shapes or stale pre-correction warnings.

All three calendar-policy manifests record `forecast_calendar_policy='day_after_history_end_v1'`. Artifact contracts reject stale metadata and tuning rejects incompatible baselines. Pandoc is installed for report conversion. Git installation via `winget` was cancelled at the administrator prompt; Git readiness is not established. Final visual and report exports have been regenerated and verified against the corrected run. Repeat reconciliation after future source or narrative changes.

Responsible-use notice: this is an academic capstone prototype built on a historical dataset with verified public-file identity but unresolved daily coverage. It must not be presented as a live SLA-compliance system, a production capacity-planning tool, or evidence of current help-desk performance. Descriptive figures and retained backtest metrics concern historical observations ending March 2023.

Executive summary

This capstone analyzes the multi-year `data/raw/issues.csv` help-desk dataset across two integrated workstreams:

1. **Retrospective SLA-reference analysis** — comparing resolved ticket records' elapsed wall-clock resolution time against project-configured priority reference thresholds.
2. **Predictive daily ticket-volume forecasting** — comparing one-week and one-month-ahead daily-arrival forecasts across four candidate methods for capacity-planning research.

The retrospective cohort (16,735 eligible resolved tickets) met its configured reference in **19.92%** of cases. Corrected validation selects XGBoost at 7 days (validation MAE **4.181157**, test MAE **5.049222**) and weekly additive ETS at 30 days (validation MAE **4.393815**, test MAE **4.672305**, WAPE **27.506619%**). Both selected models have lower historical test MAE than the seasonal baseline. ETS has lower 7-day test MAE than XGBoost, but test results do not override validation-only selection. The primary 30-day ETS is persisted and

tracked in MLflow; the local FastAPI POC passed in-process `TestClient` checks. Neither workflow is validated against current data, approved business rules, or real operational outcomes — see [§10 Limitations and responsible use](#) before any operational reliance on these results.

This report is the single narrative document tying together the eight analysis notebooks, the generated output/artifacts, the persisted model, the MLflow experiment run, and the deployment proof of concept, and maps each section to the capstone grading rubric. The step-by-step technical detail is implemented in notebooks/ and summarized end-to-end in docs/process_flow.md. All eight notebooks were executed in the corrected local run; the results remain historical, not live-service evidence.

In summary: about 1 in 5 eligible past tickets met the configured priority reference. Corrected validation selects XGBoost for one week and ETS for one month, with historical test errors averaging about 5.05 and 4.67 tickets/day, respectively. All eight notebooks executed and 39 tests passed in the clean local submission copy using the existing environment; current-data/business-impact evidence and GitHub publication remain absent.

1. Problem understanding and framing

Element	Definition
Core retrospective task	Analyze completed <code>Ticket</code> records and compare elapsed creation-to-resolution time against project-configured priority reference thresholds.
Core predictive task	Forecast counts of exact <code>issue_type == Ticket</code> records created per UTC calendar day, evaluated separately at 7- and 30-day horizons.
SLA target	Reference-threshold attainment by mapped priority tier; unmapped (unknown) priority is excluded, not defaulted.
Forecast target	Daily <code>Ticket</code> count; inputs are restricted to past counts and calendar features only (no leakage of same-day or future information).
Primary metrics	SLA reference-met counts/rates by mapped priority; forecast validation/test MAE and RMSE.
Supporting metrics	WAPE, sMAPE, signed bias, empirical 90% interval coverage, and data-quality/coverage counts.
Explicit non-goal	This is not an individual live-ticket breach alerting system, and it is not a classifier.

Both workflows share the same source extract and the same explicit scope decisions (exact `Ticket` issue type, documented 2016+ window, UTC dates), which is why they are analyzed together rather than as two unrelated projects. Forecast decision tolerances and the service-level definitions used for the SLA workflow still require sign-off from a service owner — they are analytical references carried forward from a prior project iteration, not an approved policy.

Research acceptance criterion: after validation-only selection at each horizon, require held-out MAE below the same-weekday seasonal baseline at both 7 and 30 days, with RMSE, bias and interval coverage disclosed. The corrected run meets this historical criterion: 5.05 versus 6.41 tickets/day at 7 days and 4.67 versus 6.49 at 30 days. This is not an owner-approved operational acceptance or evidence of a business KPI.

Proposed business success criterion (not measured or owner-approved): a human-reviewed pilot should reduce overtime hours per 100 tickets by **at least 10%** versus a matched baseline, without worsening an owner-approved service-attainment rate. Approve the ticket denominator, overtime and service-clock definitions, comparison period and matching rules, costs and stop conditions, and obtain current representative data before the pilot. The historical reference-attainment rate is not an approved operational KPI; no savings or ROI is measured.

2. Data collection and understanding

Dataset listing: [Help Desk Tickets \(Mendeley Data\)](#), [Kaggle](#). Confirmed original metadata: **Mohammad Abdellatif (2025), Help Desk Tickets, version 2**, published **30 May 2025**, DOI **10.17632/btm76zndnt.2**, CC BY 4.0, [Mendeley Data](#). The working extract is `data/raw/issues.csv` — **66,691 rows, 58 fields**, excluded from Git and never modified in place. On 30 September 2026, its 20,586,538-byte size and SHA-256 matched the official Mendeley version-2 file metadata, and it was byte-for-byte equal to a freshly downloaded Kaggle version-1 `issues.csv` member. Direct Mendeley binary retrieval returned HTTP 403, so that comparison is with the publisher's digest, not a separately downloaded Mendeley CSV. The full [verification record](#) retains the public endpoints, file identifiers, hashes and comparison method.

License-label discrepancy: Kaggle metadata labels the mirror CC0, whereas Mendeley specifies CC BY 4.0. The mirror label does not establish relicensing authority. Preserve original attribution, the license link and indication of changes, resolve the mirror discrepancy, and review third-party rights and privacy before sharing derived material. Raw data remains excluded. Identity does not prove daily completeness or valid service policy.

- The source documentation describes the file as covering **January 2016–March 2023**, but the local extract's `issue_created` field also contains dates back to **2007-04-15**. The primary analysis follows the documented window and reports the mismatch as an open provenance/quality issue rather than silently excluding or including the earlier rows without comment.
- The forecast/EDA scope is the exact `issue_type == Ticket` category: **27,348 rows** on/after the 2016 scope start. This deliberately excludes epics, stories, vacations, subtasks, and other non-ticket issue types from being counted as ticket arrivals.
- A generated, complete **58-field data dictionary** records dtype, present/missing row counts, and unique-value counts for every source field (see §12 and `output/issue_helpdesk/data_dictionary.csv`). Notable gaps: 853 missing resolution dates; 46.55% of last-assignee values missing; most workflow-state duration fields are sparse. No exact full-row duplicates were found; issue IDs are unique.

2.1 Data structure

The raw source is one flat table (66,691 rows by 58 columns), grouped by role:

Group	Example columns	Role
Identity/keys	<code>id</code> , <code>issue_num</code> , <code>issue_proj</code>	Unique record and (masked) project/issue references.
Actors	<code>issue_reporter</code> , <code>issue_assignee</code> , <code>issue_contr_count</code>	Masked user identifiers; <code>issue_assignee</code> is 46.55% missing.
Classification	<code>issue_type</code> , <code>issue_priority</code> , <code>issue_status</code> , <code>issue_resolution</code>	Ticket type, priority, workflow status, resolution outcome; the primary cohort filters <code>issue_type == "Ticket"</code> and <code>issue_resolution == "Done"</code> .
Timestamps	<code>started</code> , <code>ended</code> , <code>issue_created</code> , <code>issue_resolution_date</code> , <code>last_change_date</code>	Parsed as UTC; <code>issue_created</code> drives the daily forecast series and <code>issue_resolution_date</code> drives the SLA reference duration. 853 rows have no resolution date.
Workflow-state durations	16 <code>wf_*</code> fields (<code>wf_in_review</code> , <code>wf_in_progress</code> , <code>wf_done</code> , ...)	Elapsed seconds in each named workflow state; mostly sparse, kept only as diagnostics, never substituted for the SLA clock.
Workflow-state transition counts	16 <code>wfe_*</code> fields	Count of times an issue passed through each state; always populated.
Process summary	<code>wf_total_time</code> , <code>processing_steps</code> , <code>issue_comments_count</code>	Total workflow processing time and step/comment counts; diagnostic only.

The complete, generated field-by-field reference (dtype, rows processed, present/missing counts, unique-value counts, one-line meaning/caveat for every one of the 58 columns) is

output/issue_helpdesk/data_dictionary.csv.

The **engineered structure used for forecasting** is a separate, much narrower table: one row per **calendar day** (2,628 rows), built only from `issue_created` day counts so no same-day raw field leaks into the target:

Feature group	Columns	Notes
Target	<code>ticket_count</code>	Scoped Ticket records created that UTC day; 0 for the 344 zero-ticket days.
Lag features	<code>lag_1 ... lag_28</code>	Prior 28 days' counts, computed only from history strictly before the target date.
Rolling means	<code>mean_7, mean_28</code>	7- and 28-day trailing averages, same past-only rule.
Calendar encodings	<code>weekday_sin, weekday_cos, year_sin, year_cos</code>	Cyclical day-of-week/day-of-year encodings for weekly/annual seasonality without a boundary discontinuity.

This 34-feature-plus-target table (`src/issues_capstone.py::supervised_features`) is what feeds the rolling-origin backtest, mutual-information selection, PCA, and the persisted final model.

3. Preprocessing, EDA and feature engineering

Preprocessing decisions (Notebook 01–02):

1. Parse `issue_created` with mixed fractional-second formats as UTC.
2. Restrict the forecast target to `issue_type == Ticket` and the documented 2016+ scope.
3. Aggregate to one count per UTC calendar day.
4. Fill dates without rows as zero **only** under an explicit, flagged completeness assumption (344 of 2,628 days, 13.1%, are zero-filled this way — not independently verified as true zero-arrival days).
5. Construct **past-only** lag (1–28 day), rolling-mean (7/28-day), weekday sin/cos, and annual-cycle sin/cos features. A dedicated contract test (`test_daily_lag_features_use_only_previous_history`) checks the history rule; recursive-calendar contracts and the corrected artifacts were also checked in the final 39-test local submission-copy pass.

EDA (Notebook 02): descriptive breakdowns by year, mapped priority, and weekday; resolution-duration correlations; and the retrospective SLA reference table (below). **Feature selection and dimensionality reduction:** training-only mutual-information ranking of the engineered feature set, and PCA diagnostics run only on the pre-validation training window (never refit on validation/test data, to avoid look-ahead leakage). PCA was evaluated as a candidate for the Ridge challenger; component variance and score contribution were found modest relative to the raw lag features, so PCA/MI-selected Ridge is retained as one of the four compared candidates rather than replacing the full feature set outright — see the "PCA and dimension reduction" and "feature generation" discussion cells in Notebook 02 for the full reasoning.

The resulting daily series has **2,628 calendar-day observations** (2016-01-03 to 2023-03-14), averaging **10.4 tickets/day**, ranging from zero to 88.

Missing-date sensitivity evidence

The supplementary [sensitivity summary](#) and [metrics](#) test the assumption that all 344 zero-record dates are genuine zero demand. A deliberately severe alternative treats them as unknown and replaces training-history zeros with the median of up to four earlier positive same-weekday counts within 56 days; if none exist, it uses earlier positive counts in that window. No future or previously imputed value contributes to a replacement.

ETS, XGBoost and seasonal naive are refitted at the original rolling origins. Ridge is outside this bounded check. Both histories are scored on identical positive-record targets; imputed labels are never treated as observed truth.

The alternative history contains 28,119 counts versus 27,348 recorded tickets, a hypothetical total rather than newly discovered tickets.

Model/horizon	Test MAE: original history, positive targets	Test MAE: stress history, same targets
XGBoost, 7 days	5.1241	5.1781
ETS, 30 days	4.7510	4.7897
Seasonal naive, 7 days	6.6439	6.5341
Seasonal naive, 30 days	6.6628	6.5748

The selected models retain a baseline advantage on these targets, with approximately 1.1% and 0.8% changes in their test MAEs. However, merely excluding zero-record validation targets changes the 7-day ranking: ETS 4.3398 versus XGBoost 4.4408. ETS is best among the three tested families at both horizons under the alternative history as well. The canonical 7-day winner is therefore sensitive to the evaluation population; this is an important limitation, not grounds to select on test.

This analysis is post-hoc and changes the scored population. Positive-record days can also be incomplete; excluding zero dates can introduce selection bias. Missing-day true counts remain unknown, overlapping windows are not independent, and intervals have not been recalibrated. The canonical 7-day XGBoost and 30-day ETS selections, metrics and persisted model are unchanged. Reproduce with `python -m src.generate_coverage_sensitivity` after Notebook 03.

Retrospective priority-SLA reference assessment

Mapping carried forward from the prior project: Blocker/Highest → Critical (4h), High → High (8h), Medium → Medium (24h), Low/Lowest → Low (24h); source unknown priority is excluded, not defaulted. Duration is `issue_resolution_date - issue_created` in elapsed UTC wall-clock hours — not confirmed as a business-hours or pause-adjusted SLA clock. The eligible cohort is exact Ticket records, documented period, resolution Done, known mapped priority, and valid nonnegative elapsed duration.

16,735 records met eligibility; **3,333 (19.92%)** met the configured reference.

Mapped priority	Reference	Assessed	Met	Met rate
Critical	4h	2,124	524	24.7%
High	8h	3,146	576	18.3%
Medium	24h	11,161	2,161	19.4%
Low	24h	304	72	23.7%

These priority-specific counts are the shared retrospective baseline reported consistently across Notebook 02, the technical/business decks, and this report; they are not a causal or fairness comparison across priority tiers (see §5).

4. Model implementation and comparison

Experiment design (Notebook 03): separate 7-day and 30-day horizons, each using weekly-spaced **expanding-window rolling origins**, a full 365-day validation period for model selection, and a later, disjoint 365-day test period. The 7-day experiment yields 52 validation and 51 test origins; the 30-day experiment yields 48 origins in each split (each forecast window must fit entirely inside its split). The 30-day output is a **daily path** for the next 30 days, not a single monthly total; its weekly-spaced origins overlap, so pooled test metrics summarize many historical origin/lead-day forecasts, not 48 independent months.

Candidates, selected separately per horizon on validation MAE only (test data is never used for selection):

- Same-weekday-last-week seasonal-naive baseline.
- Additive weekly ETS (exponential smoothing).
- XGBoost autoregression on the preceding 28 daily counts plus calendar features.
- Ridge regression with training-only mutual-information feature selection and PCA.

Corrected validation selection: XGBoost is selected at 7 days and ETS at the primary 30-day horizon. All MAEs below are tickets/day.

Method	7-day validation MAE	30-day validation MAE	7-day test MAE	30-day test MAE
Seasonal naive	5.142857	5.411806	6.408964	6.489583
Weekly ETS	4.227266	4.393815	4.758324	4.672305
XGBoost	4.181157	4.684999	5.049222	5.421653
Feature-selected/PCA	4.323507	4.778375	5.217716	5.454248
Ridge				

ETS has lower 7-day test MAE than XGBoost; it is **not** selected at that horizon because selection is fixed by validation MAE, never test rank.

Corrected historical test results (7-day horizon):

Method	MAE	RMSE	WAPE	sMAPE	90% interval coverage
Weekly ETS (comparator)	4.76	7.40	28.66%	44.76%	87.4%
Same-weekday-last-week baseline	6.41	9.83	38.60%	52.86%	86.3%
XGBoost (validation-selected)	5.05	7.72	30.41%	49.03%	86.6%
Feature-selected/PCA Ridge	5.22	8.01	31.43%	45.79%	83.8%

Corrected XGBoost 7-day test values are MAE **5.049222**, RMSE **7.722058**, WAPE **30.412893%**, sMAPE **49.034548%**, and coverage **86.554622%**. The corrected tuning comparison has equal baseline/candidate validation MAE at each horizon (4.181157 at 7 days; 4.684999 at 30 days), so the decision is keep_baseline. This bounded comparison demonstrates no improvement, not a general optimum. Earlier improvement claims based on the invalid calendar baseline are superseded.

Corrected historical test results (30-day horizon, primary/persisted model): weekly ETS MAE **4.67 tickets/day**, RMSE **7.20**, WAPE **27.51%**, sMAPE **41.33%**, with a validation-calibrated 90% interval covering **87.7%** of test forecasts. Test MAE by lead-day band: 4.60 (days 1–7), 4.49 (8–14), 4.67 (15–21), 4.88 (22–30) — error grows modestly with lead time, as expected. The seasonal-naive baseline's error rises more steeply across the same bands (6.04 → 6.91 tickets/day), showing an aggregate baseline advantage across those bands, not superiority on every forecast. ETS metrics were not directly affected by the recursive calendar bug, and the corrected comparison retains ETS for the 30-day horizon.

The exact primary test values rounded to six decimals are MAE **4.672305**, RMSE **7.197036**, WAPE **27.506619%**, sMAPE **41.327574%**, and coverage **87.708333%**.

Metric interpretation: MAE/RMSE are in tickets/day (RMSE penalizes larger misses more); WAPE is total absolute error over total actual volume; sMAPE is a symmetric relative-error measure chosen to avoid the instability of MAPE near zero-count days. The displayed 90% interval uses a validation-only quantile of absolute residuals; selected-model test coverage (86.55% for 7-day XGBoost and 87.71% for 30-day ETS) is below the nominal 90%, so calibration is imperfect and not guaranteed to persist on new data.

Model persistence: the validation-selected primary-horizon model is refit on the **full** available history (2,628 days, 2016-01-03 to 2023-03-14) and persisted as the project's single reproducible artifact: `models/final_volume_forecast_model.joblib` plus `models/final_volume_forecast_model_metadata.json` (selected model name, validation MAE, training-data range, usage instructions, and the same limitations that apply project-wide). `persist_final_model()` in `src/issues_capstone.py` performs no retraining beyond this refit — it does not re-run model *selection*, which remains governed by the validation comparison above. A dedicated contract test (`test_final_model_artifact_is_persisted_and_reproduces_selection`) checks metadata against notebook 03 selection and passed in the local 39-test local submission-copy suite. The API serves **one persisted primary 30-day ETS artifact for every accepted horizon**. Requesting `horizon_days=7` does not load the validation-selected 7-day XGBoost; **no separate 7-day model is persisted**.

5. Critical thinking, ethical AI, and bias auditing

Data and method limitations carried through every stage: the unresolved date-range provenance mismatch; the unverified zero-arrival-day completeness assumption; the fact that the dataset ends in March 2023 (over two years stale relative to any live claim); and the fact that the elapsed-duration SLA clock is not confirmed to match a business-hours/pause-adjusted policy.

Explainability (Notebook 04): SHAP values are computed only for the XGBoost model fitted on pre-validation data and evaluated on validation dates only. XGBoost is selected at 7 days and a challenger at 30 days; these diagnostics do not explain every recursive multi-step forecast or the primary ETS model. Leading SHAP contributions are lags 28, 21, and 14 days. Correlated lag features can share importance; feature importance/SHAP here describe statistical model behavior, not causal real-world demand drivers.

Direct primary-model explanation: the [ETS component figure](#), [residual/error figure](#) and [diagnostic summary](#) explain the saved 30-day ETS itself, without refitting it. The final level is 20.0965, daily trend increment 0.000713, and effective weekly seasonal offsets range from -14.9228 to +8.3789 tickets/day. Level/trend/seasonal smoothing weights are 0.04923, 0 and 0.04373. A zero trend smoothing weight preserves the fitted trend rather than setting its value to zero. Raw level and seasonal offsets depend on state normalization; they do not identify causal demand drivers.

One-step fitted values use previous level, previous trend and the lagged seasonal state, not the post-update states that already contain the current observation. Future raw values equal terminal level plus lead times trend plus the effective repeated weekly seasonal contribution. The installed library's effective forecast cycle differs from its last updated seasonal states by up to 0.3091 tickets at the forecast boundary. Both are exported explicitly and the effective cycle reconstructs all 30 saved-model predictions; model behavior has not been changed.

Full-history fitted residuals have MAE 3.6749 and RMSE 5.5517, with mean actual-minus-fitted error 0.00012. A near-zero mean does not establish adequacy: lag-1 ACF is 0.1346 and lag-7 ACF is -0.00135; the cumulative Ljung-Box p-value through lag 7 is approximately 1.55e-12. Remaining serial structure is indicated. The test uses `model_df=0` without fitted-parameter or multiple-testing adjustment, so it is exploratory rather than a calibrated post-estimation certification.

These states and residuals come from a fit on all history, including dates in the earlier test. **Held-out performance is reported separately** from the saved rolling forecasts: MAE 4.6723, RMSE 7.1970 and signed bias **+0.7087 tickets/day**, indicating average underprediction. The 1,440 origin/lead pairs cover 48 origins and reuse 359 target dates. Reproduce with `python -m src.generate_ets_diagnostics` after Notebook 03.

Fairness/bias audit (Notebook 05): the source contains masked project/reporting identifiers, not approved protected demographic attributes, so **protected-group fairness cannot be assessed**. The audit instead reports descriptive operational-group breakdowns (by priority, year, and project) with Wilson confidence intervals for small-group reference rates, explicitly not presented as a demographic fairness certification. The SLA reference comparison is partly defined by priority-specific thresholds, so between-priority attainment differences are a

designed asymmetry, not by itself evidence of unfair treatment. The daily forecast is a single aggregate series, so it cannot reveal whether any one queue, site, or shift is under-served; if approved operational segments become available in the future, signed bias, error, and interval coverage should be audited per segment with small-group uncertainty retained, and human capacity-planner review should remain in the loop.

Generative-AI scope decision: no generative/LLM component is used anywhere in the analysis, forecast, or deployment path. Notebook 06's "dashboard/GenAI" deliverable is a machine-readable data contract and template-based summary table, not an LLM call — this keeps every number in the SLA and forecast outputs reproducible and auditable, which a generative model would not guarantee, at the cost of natural-language flexibility.

6. Deployment readiness, monitoring, and experiment tracking

Notebook 07 and `src/app.py` together answer "what would it take to operate this model," without claiming operational readiness has been reached (see §10).

Experiment tracking (MLflow). `log_experiment_to_mlflow()` registers the notebook 03 selection outcome and the persisted `models/` artifact as one tracked MLflow run — no retraining occurs. It logs:

- **Params:** model name (ETS (weekly additive)), selection rule (minimum validation MAE only, test unused for selection), primary horizon (30 days), training window (2016-01-03 to 2023-03-14, 2,628 rows).
- **Metrics:** validation MAE (4.39) and the primary-horizon test metrics (MAE 4.67, RMSE 7.20, WAPE 27.51%, sMAPE 41.33%, 90% interval coverage 87.7%) — identical to the values reported in §4, by construction.
- **Artifacts:** the `.joblib` model file, its metadata JSON, and the notebook 03 selection JSON.

Tracking metadata is stored in a local SQLite store (`mlflow.db`, tracking URI `sqlite:///mlflow.db`) with run artifacts under `mlruns/`; both are gitignored and regenerated by re-running this step — they are a local index over local artifact files (`models/`, `output/issue_helpdesk/`), not a unique source of truth. View a run locally with:

```
mlflow ui --backend-store-uri "sqlite:///mlflow.db"
```

A machine-readable summary of the logged run is also written to `output/issue_helpdesk/mlflow_run_summary.json`; the corresponding check is `test_mlflow_run_summary_matches_persisted_model`. The corrected run refreshed this summary and the corresponding local contract passed.

Historical monitoring. `run_monitoring_plan()` produces `forecast_historical_monitoring.csv` — a demonstration of the monitoring checks (signed bias, MAE/RMSE/WAPE, interval coverage over time) that a real pilot would need, not a live alerting system. See `docs/deployment_monitoring.md` for the full data contract, pre-pilot checklist, and stop conditions.

Deployment app (proof of concept). `src/app.py` is a minimal FastAPI service designed to load the same persisted artifact — no retraining, no new metrics — and serve it over HTTP. The real model and API passed both in-process `TestClient` checks and an actual localhost HTTP demonstration: health OK, seven dated ETS points starting 2023-03-15, and HTTP 422 for an invalid horizon. The temporary server was stopped afterward; no currently running endpoint, Docker deployment or hosted service is claimed.

Endpoint	Method	Purpose
/health	GET	Metadata-presence check; does not prove successful model loading or inference. A successful /forecast response is also required.

Endpoint	Method	Purpose
/model/metadata	GET	Returns the persisted model's metadata (name, validation MAE, training range, usage/limitation notes) verbatim from <code>models/final_volume_forecast_model_metadata.json</code> .
/forecast? horizon_days=N	GET	Returns a daily forecast path for the next N days (1–90, default 30) starting the day after the end of the stored history. Every response includes a caveat field repeating the historical-backtest limitation.

Run it locally with:

```
python -m pip install -r requirements.txt
uvicorn src.app:app --reload
# then, e.g.:
curl http://127.0.0.1:8000/health
curl http://127.0.0.1:8000/forecast?horizon_days=7
```

`test_deployment_app_serves_persisted_model_forecast` exercises `/health` and `/forecast` (including a 422 validation case for `horizon_days=0`) through FastAPI's `TestClient` and passed in the final 39-test local submission-copy suite. **This is intentionally a local proof of concept, not a hosted/production service:** there is no authentication, no autoscaling, no scheduled retraining trigger, and no current-data feed. The source implements a serving POC, not evidence of a currently functioning endpoint or operational reliability.

Release gates. Before any of this could move beyond a local proof-of-concept, historical backtests and local API checks are not enough. The following operational gates remain open: (1) the source date-range/zero-day questions in §2–3 are resolved; (2) the SLA mapping and elapsed-duration clock are approved by a service owner; (3) the model is re-validated on current, representative data and a genuinely future holdout; (4) approved operational segments exist if a segmented or fairness-audited forecast is required; and (5) a human-reviewed pilot with real staffing outcomes exists before any ROI claim. The tracked MLflow run and the monitoring trend are necessary evidence for that future decision, not sufficient evidence on their own.

7. Reproducibility guide

This section is the single authoritative path to regenerate every number, artifact, and endpoint in this report from a local source copy. It intentionally repeats details from `README.md` and `docs/process_flow.md` so this report is self-contained.

All eight notebooks completed in the actual project `.venv`. Final checks passed 39/39 tests in a clean local submission copy using that existing environment, without raw CSV, `mlflow.db`, or `editor/cache/archive` folders. The copy's tests verify packaged artifacts; they do not rerun the raw-data analysis without its input. This was not a Git clone, new environment, Docker or cloud CI run. The earlier SciPy DLL block no longer reproduces; no policy bypass was used.

7.1 Environment setup

```
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Place the supplied source file at `data/raw/issues.csv` (never committed to Git; excluded via `.gitignore`).

7.2 Regenerate the analysis in order

Calendar-correction revalidation: with verified Notebook 01/02 inputs, rerun **Notebook 03 first**, then dependent notebooks **04–08** and their exports. For a clean reproduction or changed source/scope, start at Notebook 01. Review authored README, report and narrative metrics against corrected CSVs and selection/tuning records before exporting; format conversion does not update manually written numbers.

Earlier notebook revalidation notices were removed after all eight notebooks executed and artifact/full-suite verification passed. The final suite now has 39 passing tests, including two portable fixture cases. Notebook 03/04 explanations and the generator template now reflect corrected horizon-specific selection and superseded invalid-baseline tuning claims. Tests require `forecast_calendar_policy='day_after_history_end_v1'`; all three manifests carry the marker. For future changes, keep any stale-result notices until artifacts and the full suite are verified. The poster displays **RE-RUN REQUIRED** for missing policy metadata; do not manually relabel old artifacts to bypass that safeguard.

Run all eight notebooks **in numerical order** — each is idempotent and re-writes its own artifacts under `output/issue_helpdesk/`:

```
01_data_understanding.ipynb
02_eda_feature_engineering.ipynb
03_model_training_comparison.ipynb
04_explainability_shap_lime_pdp.ipynb
05_bias_audit_fairness_evaluation.ipynb
06_genai_dashboard_mlops.ipynb
07_deployment_monitoring_mlflow.ipynb
08_business_analytics_reporting_roi.ipynb
```

Critical requirement: execute every notebook with the project's own Jupyter kernel — `helpdesk-project-venv` (the `.venv` created above registered as a kernel) — not the default `python3/Anaconda` kernelspec that each notebook nominally declares. A joblib-pickled model or a different `scipy/statsmodels` build under a different interpreter is not guaranteed to unpickle correctly elsewhere. From the command line this looks like:

```
python -m ipykernel install --user --name helpdesk-project-venv --display-name "Python (helpdesk-project-venv)"
jupyter nbconvert --to notebook --execute --inplace --ExecutePreprocessor.kernel_name=helpdesk-project-venv notebooks\01_data_understanding.ipynb
:: ... repeat through notebooks\08_business_analytics_reporting_roi.ipynb
```

For source profiling/forecasting and deck packaging only, from the project root:

```
python presentations\run_volume_forecast.py
python presentations\generate_presentation.py
```

refreshes the forecast/data artifacts and rebuilds both presentation decks without opening Jupyter directly. This shortcut does not replace dependent notebook execution, authored-metric review or full-suite verification.

7.3 Regenerate this report

Pandoc is installed in this workspace at `%LOCALAPPDATA%\Pandoc\pandoc.exe`. A new terminal may be needed to see the updated `PATH`; alternatively, add its directory for the current PowerShell session before running the exporter:

```
$env:PATH = "$env:LOCALAPPDATA\Pandoc;$env:PATH"
pandoc --version
```

```
python -m pip install -r requirements-report.txt
python src\generate_final_report.py
```

writes `output/issue_helpdesk/reports/final_capstone_report.docx` and `.pdf` from this same Markdown source (`final_capstone_report.md`), so all three formats stay in sync after a successful export. Pandoc is a system dependency, not a Python package. PDF export also requires a compatible Chromium-family browser; use `--browser` with its executable path if automatic detection fails. The exporter uses an isolated browser profile, verifies the generated PDF and fails explicitly if a required browser or PDF is missing. `--skip-pdf` requests an intentional DOCX-only export, not a successful PDF export. These checks verify packaging, not corrected forecast results.

7.4 Run the deployment app

```
uvicorn src.app:app --reload
```

then request `http://127.0.0.1:8000/health`, `http://127.0.0.1:8000/model/metadata`, and `http://127.0.0.1:8000/forecast?horizon_days=30`. The app requires `models/final_volume_forecast_model.joblib` and the canonical `daily_ticket_counts.csv` (produced by notebook 01/03) to already exist — run step 7.2 first if either is missing.

7.5 Run in Docker

```
# Lightweight contract-test image (requirements-test.txt only)
docker build -t issues-capstone .
docker run --rm issues-capstone

# Full analysis image (requirements.txt; needed to regenerate notebooks/app)
docker build --target analysis -t issues-capstone-analysis .
docker run --rm -v ${PWD}\data\raw:/app/data/raw -v ${PWD}\output:/app/output issues-capstone-
analysis `
python presentations\run_volume_forecast.py
```

The lightweight test image intentionally does not install `joblib`, `fastapi`, or `mlflow`; the three tests that need them (`test_final_model_artifact_is_persisted_and_reproduces_selection`, `test_deployment_app_serves_persisted_model_forecast`, and any future equivalents) may **skip** in that environment. A lightweight run does not validate the full analysis. Docker and GitHub Actions were not executed; the final 39-test result is from a clean local submission copy using the existing `.venv`, not a container or cloud run.

7.6 Run the full test suite

```
python -m unittest discover -s tests -v
```

The final suite passed **39/39 tests in a clean LOCAL submission copy** using the existing `.venv`. Two portable fixture cases were added to the suite. The copy omitted raw CSV, `mlflow.db`, and `editor/cache/archive` folders; it was not a Git clone or a new dependency environment. All eight notebooks had already executed with the real kernel and source data in the working project. Final tests include calendar-policy artifact contracts and API checks with the persisted model, not only estimator doubles. This is local package validation, not Docker/cloud CI or a raw-data rerun inside the submission copy.

8. Final presentation and communication

- **Technical deck:** `presentations/technical_capstone.pptx` (also packaged at `presentations/technical_presentation.pptx`), built by `src/generate_capstone_decks.py::build_all`.
- **Business deck:** `presentations/business_capstone.pptx`.

- **Authored technical narrative:** presentations/technical_presentation.md (a slide-by-slide companion, cross-referenced from this report rather than duplicated verbatim).
- **This report** is the consolidated, rubric-aligned written deliverable tying together all of the above; see `src/generate_final_report.py` for the reproducible DOCX/PDF export described in §7.3.

Both decks separate validation from test results, disclose forecast uncertainty and interval coverage, and explicitly withhold ROI/current-service claims that the available data cannot support.

Technical-slide clipping has been fixed, and corrected deck/poster exports have been regenerated without stale revalidation warnings. The canonical technical/business decks contain **12/10 slides**, respectively, with no off-slide shapes. Final report DOCX and PDF have been regenerated from the corrected source using installed Pandoc and an isolated Edge profile, reflecting the XGBoost 7-day/ETS 30-day results. Final visual/instructor review and publication remain separate from the successful analysis/test run.

All active presentation files are under root presentations. The superseded archived deck, obsolete classifier artifacts and unused sample/legacy Python files have been removed. Current analysis outputs, the virtual environment, caches and local MLflow records are retained.

9. GitHub profile and repository readiness

The local workspace is organized as a reusable pipeline: eight sequentially-runnable notebooks, a shared `src/issues_capstone.py` module, `configs/project_config.yaml`, `full requirements.txt` and pinned `lightweight requirements-test.txt` dependencies, a Docker test/analysis multi-stage build, a GitHub Actions CI workflow, and this report. Raw data is excluded from Git throughout.

Source identity verified; coverage and mirror licensing still open: local bytes match the downloaded Kaggle v1 file and Mendeley v2's published digest/size (§2). The source description's early-date discrepancy is not resolved by identity. The Kaggle CC0 label conflicts with original CC BY 4.0; retain the original attribution and license. Keep raw data excluded and review derived content, third-party rights and privacy.

Outstanding: this project has not yet been published to a public GitHub repository from this workspace. No public repository URL or public upload has been established. Corrected local validation is complete, but final instructor/content review and publication review remain. Git installation via `winget` was cancelled at the administrator prompt; no `.git`, remote or publication was created. No Docker CLI is available, so container execution is unverified. Neither Git readiness nor a GitHub Actions run is claimed.

10. Limitations and responsible use

This project is an academic capstone prototype. It must **not** be used to:

- certify current or contractual SLA compliance;
- make live staffing, hiring, or budget decisions;
- represent, alone, evidence of protected-group fairness (no approved protected attributes exist in this dataset); or
- represent current help-desk conditions (the source data ends March 2023).

Before any real-world application:

1. Recheck source identity after any input change with `python -m src.verify_dataset_source`; add `--download-kaggle` for a fresh pinned mirror comparison. Preserve the evidence in §2, resolve the license-label discrepancy, and review privacy. Obtain provider evidence for the date-range mismatch and completeness assumption; the sensitivity test in §3 does not certify coverage.

2. Obtain service-owner approval for the SLA priority mapping, thresholds, Done-eligibility rule, and elapsed-duration clock definition, if this analysis is ever handed to a real help desk team. There is no external service owner for this academic capstone, so the mapping is used as a stated, inherited assumption throughout this report rather than verified policy.
3. Re-run the selected forecast method on current, representative data and a genuinely future holdout, with horizon-specific tolerances agreed in advance, once a newer extract becomes available. The source data ends March 2023, so this step cannot be run within the current scope; the pipeline is written to accept a refreshed data/raw/issues.csv without code changes when one exists.
4. Obtain approved operational segments (queue/site/shift) or protected attributes only if a segmented or fairness-audited analysis becomes a real requirement — do not infer or impute them. issues.csv has no such fields today, so this is future work contingent on a richer dataset.
5. Run a human-reviewed pilot and measure real staffing/service outcomes and approved costs before any ROI claim (see the "what this means" note below).

What "measure staffing/service outcomes and approved costs" means: this project provides corrected historical model comparisons; it does not prove that acting on the forecast improves anything in the real world. A credible ROI (return on investment) claim needs, from an actual pilot: a defined trial period with a human still reviewing every decision; a before/after comparison of staffing, response times, or missed-SLA counts; the approved real cost of running the pipeline and of forecast errors; and agreed benefit measures signed off by whoever owns the budget. None of that exists yet, so no ROI number is calculated or claimed anywhere in this report, the notebooks, or the decks.

Academic scope is not production acceptance

Missing demographic attributes, unmeasured ROI and lack of production deployment are not automatically academic failures. The stated academic scope is historical forecasting, retrospective reference analysis and a local deployment POC, not a demographic fairness certification, measured business intervention or live production system.

Explicit limitations, supported operational-group audits, an intentionally unfilled ROI template and human-review safeguards provide evidence of critical thinking and responsible AI practice and deserve consideration for credit. Inventing demographics, financial benefits or deployment claims would weaken the work rather than satisfy a rubric.

Missing evidence should cost marks only where the actual rubric explicitly requires it or where it is needed to substantiate a claim made here. If demographic metrics, measured ROI, an executed container or production deployment are mandatory descriptors, those requirements remain unmet. Scope disclosure is not an exemption or a guarantee of marks; the instructor's rubric and judgment remain authoritative.

11. Conclusion and next steps

This capstone materially strengthens the historical forecasting experiment relative to a prior single-month source: it supports both a one-week and a one-month rolling-origin historical validation across a multi-year daily series, four compared model families, and a persisted, tracked historical artifact and a local serving POC. Corrected validation selects XGBoost at 7 days and ETS at 30 days; both selected models have lower aggregate test MAE than the seasonal baseline. ETS's lower 7-day test error does not override validation-only selection. All eight notebooks executed and 39 tests passed in the clean local submission copy using the existing .venv; both TestClient and actual localhost HTTP checks passed. The temporary server was stopped; this is not a hosted deployment.

Immediate next steps, in priority order:

1. Review the final report DOCX/PDF, regenerated from corrected source, against the corrected CSVs and horizon-specific selection; corrected deck/poster exports are also regenerated and deck shape-bound checks passed.
2. Preserve the verified source-identity evidence, resolve the mirror license label and provider coverage questions, review permitted publication contents and obtain visual/instructor review before submission. Public repository publication is not complete (§9).
3. Obtain service-owner sign-off and current data before pursuing the proposed human-reviewed business pilot; do not claim realized ROI (§1, §10).

Conclusion: a corrected, locally reproduced historical forecasting prototype and retrospective SLA-reference analysis, with all 39 tests passing in the clean local submission copy using the existing environment. Publication and instructor review are not complete; final exports have been regenerated. Historical validation is not current operational, capacity-allocation or contractual SLA-compliance validation.

12. Rubric evidence and completeness self-review

The table below summarizes output/issue_helpdesk/capstone_completion_review.md, the project's living self-review. Corrected local validation and exports are complete; source identity and supplemental diagnostics are evidenced. Provider completeness, mirror-license resolution, instructor review and publication remain separate.

Rubric area	Current evidence	Status / remaining gap
1. Problem understanding & framing (10)	Two explicit workstreams; research baseline-advantage criterion and proposed measurable business pilot target distinguished in §1.	Academic framing evidenced. Owner approval and measured impact are operational gates, not automatic academic deductions unless required by a rubric descriptor or claim.
2. Data collection & understanding (10)	Source profiling plus direct Kaggle v1 byte equality and official Mendeley v2 digest/size match; CC BY 4.0 cited.	Identity verified. Provider date-range/completeness questions and conflicting Kaggle CC0 label remain open; privacy review is still required.
3. Preprocessing, EDA & feature engineering (10)	Scoped cohort, UTC rules, past-only features, training-only MI/PCA and causal missing-date sensitivity.	Evidence strengthened. Zero-date completeness remains unknown; 7-day validation ranking changes when zero-record targets are excluded. Canonical selection is unchanged.
4. Model implementation & comparison (20)	Corrected four-candidate comparisons, intervals/tuning, persisted primary ETS, MLflow and API checks; eight notebooks executed and 39 tests passed in a clean local submission copy using the existing environment.	Met as a historical prototype. XGBoost selected at 7 days; ETS at 30 days. Tuning ties baseline, not an improvement or general optimum. No current-data validation; not a Git clone or Docker/cloud CI run.
5. Critical thinking, ethical AI & bias auditing (20)	XGBoost explanations plus direct ETS state/residual diagnostics, sensitivity findings, Wilson intervals, operational-group audit and safeguards.	Scope-limited evidence, not fairness certification. Missing demographics/ROI/production are not automatic academic failures; explicit rubric requirements still apply (§10). Residual dependence and interval undercoverage are disclosed.
6. Final presentation & communication (10)	Corrected report/narrative and regenerated deck/poster exports; canonical technical/business decks have 12/10 slides, no off-slide shapes and no stale warnings.	DOCX/PDF regenerated from corrected source. Instructor review remains outstanding; instructor approval is not claimed.

Rubric area	Current evidence	Status / remaining gap
7. GitHub profile & upload (15)	Local organization, notebooks, configuration, pipeline, tests, Docker/CI definitions, setup guide, POC and report; raw data excluded.	Not met; not published. No public URL/upload. Git installation cancelled at admin prompt; Git readiness, content/license-label review and publication remain open.
Bonus (5)	Corrected historical time-series work, PCA/selection, MLflow, locally tested API and dual-audience decks.	Possible, not assured. Depends on final demonstration, presentation quality and evaluator judgment.

Corrected historical headline results

- Source CSV: 66,691 rows; 58 columns; exact Ticket records in the documented 2016+ scope: 27,348.
- Retrospective assessment: 16,735 eligible records; 3,333 (19.92%) met the configured references.
- Daily series: 2,628 calendar days, 2016-01-03 to 2023-03-14; 344 zero-ticket dates (13.1%) under an unverified completeness assumption.
- 7-day backtest: selected XGBoost test MAE 5.05, RMSE 7.72, WAPE 30.41%, sMAPE 49.03%; interval coverage 86.55%. ETS test MAE is lower (4.76), but XGBoost has the lowest validation MAE and remains selected.
- 30-day backtest (primary/persisted): ETS test MAE 4.67, RMSE 7.20, WAPE 27.51%, sMAPE 41.33%; 90% interval coverage 87.7%.

These corrected historical results do not establish current performance, approved SLA compliance or realized staffing value. Both validation-selected models beat the seasonal baseline on aggregate historical test MAE.

References and citation

Dataset listing: [Help Desk Tickets \(Mendeley Data\)](#), [Kaggle](#).

Confirmed original citation: Mohammad Abdellatif (2025). *Help Desk Tickets*, version 2. Mendeley Data. Published **30 May 2025**. DOI [10.17632/btm76zndnt.2](#). [Version-2 landing page](#). License: **Creative Commons Attribution 4.0 International (CC BY 4.0)**.

The source-identity comparison in §2 now connects local bytes to this published release's digest and to the downloaded Kaggle v1 member. The original [CC BY 4.0 license](#) requires attribution and indication of changes. The Kaggle CC0 label does not establish relicensing authority. Coverage, third-party rights and privacy remain distinct questions; no raw-data publication is undertaken.